

A Multi-agent Reinforcement Learning based Offloading and Resource Allocation for Vehicle Edge Computing

Huilin Yang, Xinjie Yang, Jiazhi Yang,
Ruizhe Li, Nan Ma, *Member, IEEE*, and Pei Xiao, *Senior Member, IEEE*



Abstract—In the Internet of Vehicles (IoV), vehicles have the capability to offload their computational tasks to the Mobile Edge Computing (MEC) servers in order to reduce service delay. However, the majority of existent task offloading and computational resource allocation schemes are static and lack consideration of the heterogeneous nature of tasks. Furthermore, in scenarios involving both collaborative and competitive resource utilization, there remains considerable room for performance enhancement for delay-sensitive tasks. To address these challenges, this paper proposes a novel Global Heterogeneous Multi-Agent Reinforcement Learning (GHMRL) that is an enhancement to the general MARL. In GHMRL, each vehicle and MEC server is represented by an agent, and intelligent collaboration and dynamic resource allocation are employed to balance resource usage and delay performance. In particular, GHMRL introduces a global Critic network and a local Critic network, working in a collaborative manner. The former is responsible for guiding the overall system performance to ensure the service delay performance, and the latter is responsible for MEC servers' performance to ensure the resource usage efficiency. Simulation results demonstrate that, in comparison with alternative schemes, GHMRL significantly enhances the overall system performance, particularly with regard to resource usage efficiency. Furthermore, GHMRL offers distinct advantages in balancing task delay and resource consumption under various system dynamics, making it a robust solution to address issues of resource wastage and delay violations for IoV systems.

Index Terms—Computational offloading, Multi-agent reinforcement learning, Mobile edge computing, Vehicular edge networks.

I. INTRODUCTION

Manuscript received December 10, 2025; revised 1 February 22, 2026; accepted May 17, 2026. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant U21A20448. (Corresponding author: Xinjie Yang.)

Huilin Yang, Jiazhi Yang, and Ruizhe Li are MSc students in the Faculty of Electrical Engineering and Computer Science, Ningbo University, Ningbo 315211, China (e-mail: 2411100190@nbu.edu.cn; 2211100163@nbu.edu.cn; 2411100143@nbu.edu.cn).

Xinjie Yang is with the Faculty of Electrical Engineering and Computer Science, Ningbo University, Ningbo 315211, China, and also with Ningbo Arrowdeep Internet Technology Co., Ltd., Ningbo 315021, China (e-mail: yangxinjie@nbu.edu.cn).

Nan Ma is with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: manan@bupt.edu.cn).

Pei Xiao is with the 5GIC & 6GIC, Institute for Communication Systems (ICS), University of Surrey, Guildford GU2 7XH, U.K. (e-mail: p.xiao@surrey.ac.uk).

Copyright (c) 2026 IEEE. Personal use of this material is permitted. However, permission to use this material for any other purposes must be obtained from the IEEE by sending a request to pubs-permissions@ieee.org.

IN RECENT years, significant research achievements and technological advancements in wireless communications have driven the fast development of Intelligent Traffic Systems (ITS). Communication modes have gradually evolved to support Vehicle-to-Infrastructure (V2I) and Vehicle-to-Vehicle (V2V) communications, laying the foundations for the Vehicular Ad-Hoc Network (VANET) systems [1]. At the same time, the rapid development of the Internet of Things (IoT) and Artificial Intelligence (AI) technologies has transformed vehicles into intelligent terminals [2], [3]. These intelligent terminals are equipped with in-vehicle devices similar to small computers, facilitating various VANET applications such as real-time road monitoring, vehicle-road coordination, etc [4], [5]. These applications require strict service delays and high service quality [6], [7]. However, computational power of most in-vehicle devices is limited, which has become a significant bottleneck for large-scale VANET deployment. Innovative solutions are therefore needed, for example, optimizing the hardware performance of in-vehicle devices, introducing efficient algorithms, and adopting novel network architectures such as cloud computing networks.

In previous studies, researchers proposed the use of Mobile Cloud Computing (MCC) to address the limited on-board computational capabilities for VANET systems [8]. MCC allows vehicles to fully utilize the powerful computational and efficient storage resources of cloud centers. However, since cloud servers are typically deployed at a distance, significant transmission delays can be introduced. Hence, the near-real-time response requirements of in-vehicle applications are difficult to meet. Consequently, researchers have proposed the concept of Mobile Edge Computing (MEC) that leads to vehicular edge networks [9]. In vehicular edge networks, MEC servers are typically co-sited with roadside units (RSUs) near the roads, and vehicles can offload computational tasks to MEC servers via V2I links. This approach has the potential to substantially reduce task delays and improve service quality.

The use of MEC servers for task offloading in vehicular edge networks has established a novel paradigm, attracting considerable interest from researchers [10], [11]. Ko et al. have proposed a large-scale network model based on MEC servers, investigating the scaling laws associated with communication delay, computation delay, network load, and network resources. They quantified the trade-offs between delay, network connectivity, and network stability, providing important guidance for the configuration and planning of MEC networks

[12]. Wang et al. built a multi-level MEC server architecture, consisting of the edge device layer, multi-tier MEC server layer, and cloud data center layer [13]. Data generated by edge devices can be appropriately allocated to different layers of servers for targeted processing. However, these studies primarily emphasize the integration and architecture design of MEC servers, offering limited in-depth discussions on the impact of terminal devices in the computation offloading process. It is recognized that the optimization in the computation offloading process is highly complex. For instance, recent studies have formulated the offloading decision between MEC and remote cloud as a binary integer linear programming problem, employing heuristic algorithms like Breakout Local Search (BLS) to navigate the combinatorial search space [14]. However, traditional solutions and approximation algorithms are usually based on mathematical models and expert knowledge, which limits the scalability of the models [15]. The complexity of traditional optimization solutions, coupled with their limited effectiveness, makes them unsuitable to address the dynamic requirements of MEC systems. As a result, further research on offloading node selection strategies is needed to effectively reduce delay in these dynamic environments.

In addressing these challenges, AI technologies [16] have emerged as promising solutions for the vehicular systems deployed with MEC servers. AI technologies, which facilitate optimal node selection and efficient computation for task offloading, have generated significant interest from researchers [17]–[19]. In a breakthrough study, Tang et al. proposed a distributed algorithm based on Deep Reinforcement Learning (DRL), where each device makes offloading decisions based on local observations. This approach led to substantial improvements in system performance, such as a significant reduction in packet loss rates and a lowered average delay, in handling delay-sensitive tasks under heavy load conditions [20]. Building on this, Alam et al. modeled the dynamic task offloading optimization as a bipartite graph maximum matching problem and introduced a method combining Multi-Agent Reinforcement Learning (MARL) and the Hungarian algorithm for the solution. As a result, optimal offloading action values can be found, ensuring low delay and low energy consumption. This effectively addresses task offloading issues and improves network performance in high-mobility vehicular networks [21]. Similarly, Ma et al. also modeled the task offloading optimization as an MARL problem and developed an end-to-end model to train offloading strategies. This effectively reduces delay and improves overall system performance in multiple distributed MEC server-based Internet of Vehicle (IoV) systems [22]. Concurrently, Sun et al. proposed a task offloading mechanism based on graph neural networks, and utilized a graph reinforcement learning framework for MEC systems. This reduces task delay and improves time for convergence [23]. In the study by [24], an innovative vehicle mobility-aware computation offloading scheme was proposed, where a Deep Deterministic Policy Gradient (DDPG) algorithm is utilized to train offloading decisions. When integrated with a vehicle mobility detection algorithm, it ensures the quality of vehicle-RSU communication links, as well as reduces system response delay and energy

consumption. Hence, it has been shown to be an effective solution for task offloading in urban vehicular edge computing networks.

On the other hand, AI technologies have also been extensively implemented in resource management during the computation offloading process, effectively mitigating problems such as high load, resource scarcity, and excessive task processing delays [25], [26]. Xiang et al. proposed a MARL-based spectrum access algorithm, where, based on local channel state information, each agent performs an independent learning process. Optimized spectrum resource allocation is achieved through implicit coordination, which leads to improved total throughput and packet delivery rate for vehicular users, while reducing signaling overhead [27]. Parvini et al. proposed a MARL-based resource allocation method that reduces the age of information and increases the transmission probability of cooperative sensing messages, providing a solution to the resource allocation problem in cellular IoV systems that require information timeliness [28]. Furthermore, Dong et al. proposed a joint optimization algorithm using hierarchical DRL based on network slicing to minimize queue delay and power consumption. By intelligently allocating uplink transmission resources in the environment, the algorithm effectively reduces link transmission delay while ensuring reasonable power consumption control [29]. Furthermore, [30] studied a mixed IoV satellite system and addressed the problem of satellite-assisted resource allocation. The task offloading decisions, computational resource allocation, and communication resource allocation are jointly modeled as a mixed-integer nonlinear programming problem to minimize the weighted end-to-end delay. The Lagrange multipliers and DRL are used to solve the optimization problem.

In addition, dynamic allocation of MEC server computational resources based on the type and data size of tasks can help to more efficiently achieve low-delay, high-reliability communication goals in vehicular networks [31]. Chen et al. proposed a temporal attentional deterministic policy gradient DRL method to solve the joint optimization problem of computation offloading and resource allocation in MEC-enabled IoV systems. It shows that this approach can reduce the delay in task completion and energy consumption, while also improving the convergence and stability of the training process [32]. Additionally, Nath et al. proposed a dynamic scheduling strategy that uses a deep deterministic policy gradient approach to address the challenges of dynamic caching, computation offloading, and resource allocation in MEC systems. This strategy demonstrates remarkable performance in terms of average cost, task completion time, and energy consumption [33]. Additionally, Gao et al. proposed a decentralized computational task offloading strategy based on an attention-weighted recurrent MARL, which guarantees service quality and reduces task offload delay, by dynamically allocating resource on the MEC servers [15]. Meanwhile, regarding the multi-agent architecture design, Rodrigues et al. conducted a comparative analysis between non-cooperative and cooperative offloading solutions in VEC scenarios. By designing rational agent collaboration and information-sharing mechanisms, this approach effectively improves computing energy efficiency

and reduces the overall system overhead [34]. Furthermore, to tackle stringent system constraints in such complex environments, a hybrid DRL and data-experience-driven safety layer network was proposed to solve the non-convex and non-smooth optimization problems in next-generation networks, ensuring that constraint violations are dynamically mapped back into feasible domains [35].

As evidenced by previous studies, various AI-based technologies have been proposed to address the computation offloading and resource allocation issues in complex vehicular edge networks. Despite these achievements, existing methods have not fully considered the dynamic impact of heterogeneous agents on the environment, particularly with heterogeneous network nodes (i.e. vehicles and MEC servers). The heterogeneity increases the complexity of efficient resource utilization and timely task processing, presenting numerous challenges for existing methods. To address these challenges, this paper proposes an enhanced MARL framework that aims to model collaboration and competition between heterogeneous agents and to satisfy vehicle service requirements with minimal resources. Therefore, the optimization objective is to achieve the balance between task processing delay and resource consumption, ensuring the stability, robustness, and service quality of the system. The main contributions are as follows.

- 1) In the MARL model, vehicles and MEC servers are conceptualized as heterogeneous agents. The problem of computation offloading and resource allocation is thus represented as a multi-agent problem. Vehicles employ intelligent decision-making techniques to select the most suitable MEC servers for their needs. In turn, MEC servers allocate appropriate computational resources based on the service requirements of the tasks. In addition, MEC servers implement a resource reservation strategy in order to reserve resources for future demands, thus enhancing the system's resilience and scalability.
- 2) Conventional MARL is enhanced to a novel Global Heterogeneous MARL (GHMARL) algorithm. The GHMARL categorizes agents into two distinct classes and implements different designs in the action and state spaces. As a result, agents can effectively minimize the total task processing delay and ensure the reservation of necessary resources for system stability, thereby optimizing the overall performance of vehicular edge networks.
- 3) The GHMARL implements two distinct categories of reward functions, tailored to specific agent classes. The global reward function is intended to minimize the service delay, whereas the local reward function is to optimize resource utilization. This novel structure enables the system to achieve an optimized balance between service delay and resource use.
- 4) To address the issue of overestimation bias in the function, the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm is employed to improve the performance of the training process.

The remainder of the paper is organized as follows. Section II introduces the system model. Section III describes the proposed Multi-Agent Reinforcement Learning algorithm. Section IV presents the simulation results and analysis. Finally, Section V concludes the paper. For ease of reading, the important notations used in this paper are listed in Table I.

TABLE I
LIST OF IMPORTANT NOTATIONS

Notation	Description
$\mathcal{V}$	Set of Vehicles, $\mathcal{V} = \{v_1, \dots, v_i, \dots, v_V\}$
$\mathcal{M}$	Set of MEC servers, $\mathcal{M} = \{m_1, \dots, m_j, \dots, m_M\}$
x_i	Input data size of computation task R_i
y_i	Task complexity of computation task R_i
$a_{i,j}$	Task offloading indicator of vehicle v_i
$r_{i,j}$	Data rate between vehicle v_i and MEC server m_j
$B_{i,j}$	The uplink transmission channel bandwidth
$\alpha_{i,j}$	Communication resource allocation of MEC server m_j
W_M	The communication bandwidth of MEC server
P_t	Vehicle transmission power
$DT_{i,j}$	Task transmission time from v_i to m_j
$f_{i,j}$	The computing resource for vehicle v_i
$\beta_{i,j}$	Computing resource allocation of MEC server m_j
F_M	The computing capacity of MEC server
$CT_{i,j}$	Task computation time from v_i to m_j
S_j	Resources used by the MEC server m_j

II. SYSTEM MODEL

In this paper, a vehicular edge network is considered that is deployed on a two-lane road in a typical urban environment. The network consists of a number of mobile vehicles and RSUs, and each RSU is equipped with an MEC server, as shown in Fig. 1. The road is assumed to be one-way with RSUs deployed at equal distances along the same side of the road. The set of RSU-MEC pairs is denoted as $\mathcal{M} = \{m_1, \dots, m_j, \dots, m_M\}$, with each pair assigned the same spectrum bandwidth and computational resources. For the sake of conciseness, the term of MEC server is used to represent RSU-MEC pair in the rest of the paper. The set of randomly distributed vehicles is denoted as $\mathcal{V} = \{v_1, \dots, v_i, \dots, v_V\}$, where each vehicle moves along the one-way road. The task arrival model proposed in [15] is used, where a new task is generated randomly for each vehicle at the beginning of each simulation slot according to a Poisson distribution. The task of vehicle v_i is denoted as $\{x_i, y_i\}$, where x_i represents the task data size, and y_i represents the computational complexity of the task. It is assumed that each task can only be offloaded to one MEC server, and task data is encrypted and securely transmitted. The task offloading strategy is denoted by $a_{i,j}$ ($a_{i,j} \in \{0, 1\}$), where $a_{i,j} = 1$ if the vehicle v_i offloads the task to the MEC server m_j , and $a_{i,j} = 0$ if otherwise. Therefore, the following constraints must be satisfied:

$$C1: \sum_{j=1}^M a_{i,j} = 1, i \in \mathcal{V} \quad (1)$$

After receiving a task, the MEC server m_j will allocate transmission and computational resources to vehicle v_i , denoted as $Z_{i,j} = \{\alpha_{i,j}, \beta_{i,j}\}$, where $\alpha_{i,j}$ ($0 \leq \alpha_{i,j} \leq 1$) and $\beta_{i,j}$ ($0 \leq \beta_{i,j} \leq 1$) represent the proportions of channel bandwidth and computational resources allocated, respectively.

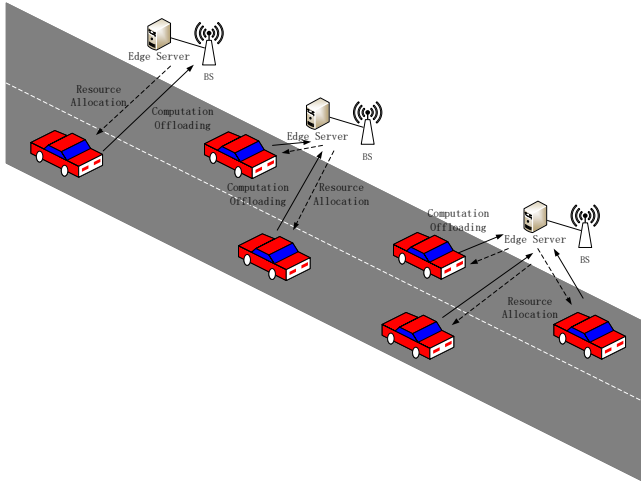


Fig. 1. The system model.

Meanwhile, the MEC server needs to reserve certain resources for future demands, denoted as $\alpha_{V+1,j}$ and $\beta_{V+1,j}$, which represent the proportions of channel bandwidth and computational resource reserved, respectively. Therefore, the following constraints must be satisfied:

$$C2: \sum_{i=0}^{V+1} \alpha_{i,j} \leq 1, j \in M \quad (2)$$

$$C3: \sum_{i=0}^{V+1} \beta_{i,j} \leq 1, j \in M \quad (3)$$

A. Transmission delay

The task delay consists of transmission delay and computation delay, and the first considers the time required for the task to be offloaded from the vehicle to the MEC server. Since the amount of result data returned to the vehicle after the MEC completes the task is relatively very small, the downlink transmission delay can be neglected.

It is assumed that the vehicular edge network is based on the international standard of NR-V2X (New Radio-Vehicle to Everything). Therefore, the system employs orthogonal frequency division multiple access (OFDMA) technology and allocates uplink channels for data offloading for each vehicle. Due to the orthogonality of the frequency divided channels, inter-channel interference can be neglected. Therefore, the uplink data transmission rate $r_{i,j}$ from vehicle v_i to MEC server m_j is expressed as:

$$r_{i,j} = B_{i,j} \log_2 \left(1 + \frac{P_t (d_{i,j})^{-c}}{\sigma^2} \right) \quad (4)$$

where $B_{i,j} = W_M \alpha_{i,j}$ represents the uplink channel bandwidth allocated, W_M is the total channel bandwidth of the MEC server. P_t is the transmission power of the vehicle. $(d_{i,j})^{-c}$ denotes the channel gain, where $d_{i,j}$ is the distance between vehicle v_i and MEC server m_j , and c is the path loss

factor. The variance of white noise in a complex Gaussian channel is denoted by σ^2 .

Therefore, the uplink transmission delay between vehicle v_i and MEC server m_j can be expressed as:

$$DT_{i,j} = \frac{x_i}{r_{i,j}} \quad (5)$$

B. Computation delay

Upon receiving a task, the MEC server allocates a proportion of computational resources, as follows:

$$f_{i,j} = \beta_{i,j} F_M \quad (6)$$

where F_M is the total computational resources of the MEC server. Therefore, the computation delay of the task is:

$$CT_{i,j} = \frac{y_i}{f_{i,j}} \quad (7)$$

The total delay of the task is:

$$T_{i,j} = DT_{i,j} + CT_{i,j} \quad (8)$$

The total resources consumed by MEC server m_j consists of bandwidth resources and computational resources, and is expressed as:

$$S_j = w_1 \cdot \sum_{i=1}^V \alpha_{i,j} W_M + w_2 \cdot \sum_{i=1}^V \beta_{i,j} F_M \quad (9)$$

where w_1 and w_2 are constants.

C. Optimization problem formulation

The objective of the optimization problem is to minimize the total delay of all tasks while using the least amount of resources and, at the same time, reserving resources for future demands for MEC servers. The goal is to find an optimal balance between the delay performance and resource consumption and to improve the system flexibility. This objective can be achieved by selecting appropriate MEC servers for offloading computational tasks and allocating their resources. Therefore, the optimization problem can be modeled as follows:

$$P1: \min_{a, \alpha, \beta} \sum_{i=1}^V \sum_{j=1}^M a_{i,j} \cdot T_{i,j} + w_3 \cdot \sum_{j=1}^M S_j \quad (10)$$

$$\text{s.t. } C1 - C3, C4: a_{i,j} \in \{0, 1\}$$

where w_3 is a constant coefficient that converts resource usage into delay. Note that the coefficients w_1 , w_2 , and w_3 serve as scaling factors to eliminate physical dimensions and align distinct metrics to a unified scale. Since system delay and resource capacities operate on significantly different numerical magnitudes, large resource values could disproportionately dominate the objective function. Therefore, the coefficient of the primary optimization metric is implicitly set to 1, and the remaining coefficients are determined based on their corresponding powers of 10. $C1$ and $C4$ indicate that each vehicle can choose only one MEC server for task offloading. $C2$ and $C3$ represent that the total bandwidth and computational resources allocated by the MEC server to the tasks must not exceed its maximum bandwidth and computational resources.

As can be seen, the optimization problem is non-convex and NP-hard [1], with high complexity, making it difficult to solve using traditional methods.

III. GLOBAL HETEROGENEOUS MULTI-AGENT REINFORCEMENT LEARNING

Due to the challenging nature of problem $P1$ when approached through conventional optimization methodologies, and given the established efficacy of DRL [1], the MARL is considered to solve the problem. To address the potential inconsistency in the impact of actions by heterogeneous agents on the environment state, an enhanced version of MARL, namely Global Heterogeneous MARL (GHMARL) algorithm, is proposed. In the algorithm, agents are defined as two complementary agents. Specifically, based on their attributes in VANET, vehicles are defined as active agents and MEC servers are defined as passive agents. Active agents execute actions based on their strategies, while passive agents make reactive decisions based on the actions taken by active agents. The purpose of this design is to avoid the instability in the environmental state that could result from the simultaneous actions of heterogeneous agents and to enhance the learning efficiency and stability of the entire system. The subsequent sections will provide a detailed description of the system state, actions, and reward functions in the multi-agent model.

A. Environment

1) *State*: For more convenient and effective modeling and analysis of the system, in this paper, the continuous time is divided into discrete time slots. The state of vehicle v_i at time slot t is denoted as:

$$s_{vi}(t) = \{g_i(t), x_i(t), y_i(t)\} \quad (11)$$

where $g_i(t)$ is the position of the vehicle at time slot t , $x_i(t)$ is the data size of the task for vehicle v_i at time slot t , and $y_i(t)$ is the corresponding computational complexity.

Similarly, the state of MEC server m_j at time slot t is represented as:

$$s_{mj}(t) = \{G_j(t), X_j(t), Y_j(t)\} \quad (12)$$

that includes the following:

- $G_j(t) = (g_{1,j}(t), \dots, g_{i,j}(t), \dots, g_{V,j}(t))$
- $X_j(t) = (x_{1,j}(t), \dots, x_{i,j}(t), \dots, x_{V,j}(t))$
- $Y_j(t) = (y_{1,j}(t), \dots, y_{i,j}(t), \dots, y_{V,j}(t))$

where $g_{i,j}(t) = g_i(t)$, $x_{i,j}(t) = x_i(t)$, $y_{i,j}(t) = y_i(t)$ are the position of the vehicle, the size and computational complexity of the task, respectively, if vehicle v_i selects MEC server m_j to offload. Otherwise, $g_{i,j}(t) = 0$, $x_{i,j}(t) = 0$, $y_{i,j}(t) = 0$.

2) *Action*: The action space of vehicle v_i at time slot t is denoted as:

$$a_{vi}(t) = (a_{i,1}(t), \dots, a_{i,j}(t), \dots, a_{i,M}(t)) \quad (13)$$

if the vehicle v_i selects the MEC server m_j for task offloading based on its current state, then $a_{i,j}(t) = 1$; otherwise, $a_{i,j}(t) = 0$.

The communication bandwidth allocation action space of MEC server m_j at time slot t can be represented as:

$$\alpha_j(t) = (\alpha_{1,j}(t), \dots, \alpha_{i,j}(t), \dots, \alpha_{V,j}(t), \alpha_{V+1,j}(t)) \quad (14)$$

where $\alpha_{i,j}(t)$ is the proportion of bandwidth resource allocated by MEC server m_j to vehicle v_i . If vehicle v_i does not select MEC server m_j for offloading, then $\alpha_{i,j}(t) = 0$. $\alpha_{V+1,j}(t)$ is the proportion of bandwidth reserved by the MEC server for future demands.

Similarly, the computational resource allocation action space of MEC server m_j at time slot t is represented as:

$$\beta_j(t) = (\beta_{1,j}(t), \dots, \beta_{i,j}(t), \dots, \beta_{V,j}(t), \beta_{V+1,j}(t)) \quad (15)$$

where $\beta_{i,j}(t)$ is the proportion of computational resource allocated by MEC server m_j to vehicle v_i . If vehicle v_i does not select MEC server m_j for offloading, then $\beta_{i,j}(t) = 0$. $\beta_{V+1,j}(t)$ is the proportion of computational resources reserved by the MEC server for future demands.

3) *Reward*: The reward function for vehicle v_i can be expressed as:

$$r_{vi}(t) = -a_{i,j}(t) \cdot T_{i,j}(t) \quad (16)$$

where $T_{i,j}(t)$ is the task execution time of vehicle v_i at time slot t when unloading the task to MEC server m_j , including both transmission and processing delays. The optimization goal is to minimize the delay, so the reward should be negatively correlated with the delay.

The reward function of the MEC server m_j is designed as:

$$r_{mj}(t) = -w_3 \cdot S_j(t) \quad (17)$$

where $S_j(t)$ is the total resource used by the MEC server to complete all tasks in time slot t , including communication bandwidth and computational resources. The optimization goal of the MEC server is to use minimum amount of resources to meet task delay requirements, so the reward function should be negatively correlated with the resources used.

The system reward function is defined as the sum of the vehicle's reward function and the MEC server's reward function. The goal of the algorithm design is to maximize the cumulative system reward, which is equivalent to the optimization objective of the original problem $P1$.

The joint state, joint action, and joint reward of the vehicle and MEC server are represented as follows:

$$\mathbf{s}_v = \{s_{vi}(t)\}, \mathbf{a}_v = \{a_{vi}(t)\}, \mathbf{r}_v = \{r_{vi}(t)\} \quad (18)$$

$$\mathbf{s}_m = \{s_{mj}(t)\}, \mathbf{a}_m = \{\alpha_j(t), \beta_j(t)\}, \mathbf{r}_m = \{r_{mj}(t)\} \quad (19)$$

For simplicity, s_{vi} , a_{vi} , r_{vi} are used to represent the state, action, and reward of vehicle v_i in the current time slot, and s'_{vi} , a'_{vi} , r'_{vi} to represent the state, action, and reward of vehicle v_i in the next time slot. Similarly, s_{mj} , a_{mj} , r_{mj} are used to represent the state, action, and reward of MEC server m_j in the current time slot, and s'_{mj} , a'_{mj} , r'_{mj} to represent the state, action, and reward of the MEC server in the next time slot.

B. Network Architecture

Contrary to conventional MARL algorithms, a centralized global Critic network architecture is used for proposed GH-MARL algorithm. The novelty of this approach lies in its ability to integrate the states and actions of all agents as input parameters, efficiently estimate the global reward, and disseminate this estimate among the agents, thereby ensuring minimal delay.

Each agent is equipped with an Actor network to facilitate real-time decision making. The vehicle's objective is to ensure minimal delay, therefore, it employs the global Critic network as its own Critic network. In contrast, the passive agent, that is, the MEC server, aims to optimize resource utilization. Consequently, it employs a local Critic network that exclusively accepts the agent's local observations and action outputs to ensure precise prediction of the local reward. This, in turn, guarantees minimal resource consumption while meeting the delay constraints. In this algorithm, agents make decisions based only on the information perceived on their own, without the need of getting other agents' information. Therefore, a distributed execution among the agents can be achieved. Moreover, it is worth noticing that the specific reserved resource portions are determined by a dynamic equilibrium between the global and local Critics. Guided by the local Critic based on the reward formulation in (17), the MEC server is incentivized to maximize reservation to minimize resource consumption penalties. However, excessive reservation reduces the computational capacity available to vehicles, triggering severe delay penalties evaluated by the global Critic. The optimal reservation ratios are ultimately generated by the Actor network, which converges via policy gradient descent driven by the joint gradients of both Critics.

Active and passive agents have a main Actor network π_{vi} , π_{mj} and a target Actor network π'_{vi} , π'_{mj} , with parameters θ_{vi} , θ_{mj} and θ'_{vi} , θ'_{mj} , respectively. The global Critic network consists of a main Critic network Q_{ω_i} and a target Critic network $Q_{\omega'_i}$, with parameters ω_i and ω'_i , respectively. The local Critic network of the passive agent also contains a main Critic network Q_{ϕ_j} and a target Critic network $Q_{\phi'_j}$, with parameters ϕ_j and ϕ'_j , respectively.

The framework of the proposed GHMARL algorithm is shown in Fig. 2. During the training process, the global Critic network is trained in a centralized manner, while the Actor and Critic networks of each agent are trained independently in a distributed manner. The active agent receives the state s_{vi} through its Actor network, and based on its policy, takes the corresponding action a_{vi} to interact with the environment. The passive agent, in turn, observes the state s_{mj} based on the action a_{vi} taken by the active agent and takes the corresponding action a_{mj} according to its policy. The environment provides rewards r_{vi} and r_{mj} based on the actions taken by the agents. The data center stores the joint state, joint action, and joint reward of all agents in the replay buffer D .

C. Network training

Network training consists of three steps. First, the global Critic network is updated. Then, the Actor network of the

active agent is updated. Finally, the Critic network and Actor network of the passive agent are updated. The goal of the network training process is to maximize the network outputs Q_{ω_i} and Q_{ϕ_j} , which are defined as:

$$Q_{\omega_i} = \mathbb{E} [r_{vi} + \gamma \mathbb{E} [Q_{\omega_i}(s'_v, s'_m, a'_v, a'_m)]] \quad (20)$$

$$Q_{\phi_j} = \mathbb{E} [r_{mj} + \gamma \mathbb{E} [Q_{\phi_j}(s'_{mj}, a'_{mj})]] \quad (21)$$

where the discount factor $\gamma \in [0, 1]$ is used to adjust the future rewards, $a'_v = \{a'_{vi}\}$, $a'_{vi} = \pi'_{vi}(s'_{vi})$, $a'_m = \{a'_{mj}\}$, $a'_{mj} = \pi'_{mj}(s'_{mj})$.

The data center randomly selects a batch of data from the replay buffer D to calculate Q_{ω_i} of the global Critic network and updates the parameters ω_i of the global Critic network by minimizing the loss function. The loss function of the global Critic network is defined as:

$$L(\omega_i) = \mathbb{E} [(Q_{\omega_i}(s_v, a_v, s_m, a_m) - y_i^g)^2] \quad (22)$$

where y_i^g is the target value for the global Critic network and is defined as:

$$y_i^g = r_{vi} + \gamma \cdot Q_{\omega'_i}(s'_v, a'_v, s'_m, a'_m) \quad (23)$$

Similarly, the data center randomly selects a batch of data from the replay buffer D to compute the Q_{ϕ_j} of the local Critic network and updates the parameters ϕ_j of the local Critic network by minimizing the loss function. The loss function for the local Critic network is defined as:

$$L(\phi_j) = \mathbb{E} [(Q_{\phi_j}(s_{mj}, a_{mj}) - y_j^l)^2] \quad (24)$$

where y_j^l is the target value for the local Critic network and is defined as:

$$y_j^l = r_{mj} + \gamma \cdot Q_{\phi'_j}(s'_{mj}, a'_{mj}) \quad (25)$$

The active agent uses the Deterministic Policy Gradient (DPG) to train its policy network. The DPG is defined as:

$$\nabla_{\theta_{vi}} \mathcal{J}_{vi} = \mathbb{E} [\nabla_{\theta_{vi}} \pi_{vi}(a_{vi}|s_{vi}) \nabla_{a_{vi}} Q_{\omega_i}(s_v, a_v, s_m, a_m)] \quad (26)$$

The passive agent updates its policy network through the global Critic network and the local Critic network. Its policy gradient is defined as:

$$\begin{aligned} \nabla_{\theta_{mj}} \mathcal{J}_{mj} = & \underbrace{\mathbb{E} [\nabla_{\theta_{mj}} \pi_{mj}(a_{mj}|s_{mj}) \nabla_{a_{mj}} \mathbb{E} [Q_{\omega_i}(s_v, a_v, s_m, a_m)]]}_{GlobalCritic} \\ & + \underbrace{\mathbb{E} [\nabla_{\theta_{mj}} \pi_{mj}(a_{mj}|s_{mj}) \nabla_{a_{mj}} Q_{\phi_j}(s_{mj}, a_{mj})]}_{LocalCritic} \end{aligned} \quad (27)$$

Despite the efficiency of the algorithm design mentioned above, the approximation errors inherent in the simulation function may result in overestimation and local optimization that lead to suboptimal strategies. To address this limitation, the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm is adopted to enhance the update of the agent's policy network. Firstly, truncated double Q-learning is employed to mitigate the overestimation of the value network. Secondly, noise is introduced into the target policy network to ensure smoother output. Finally, reduced update frequencies are used for the policy network and target networks. Specifically, the

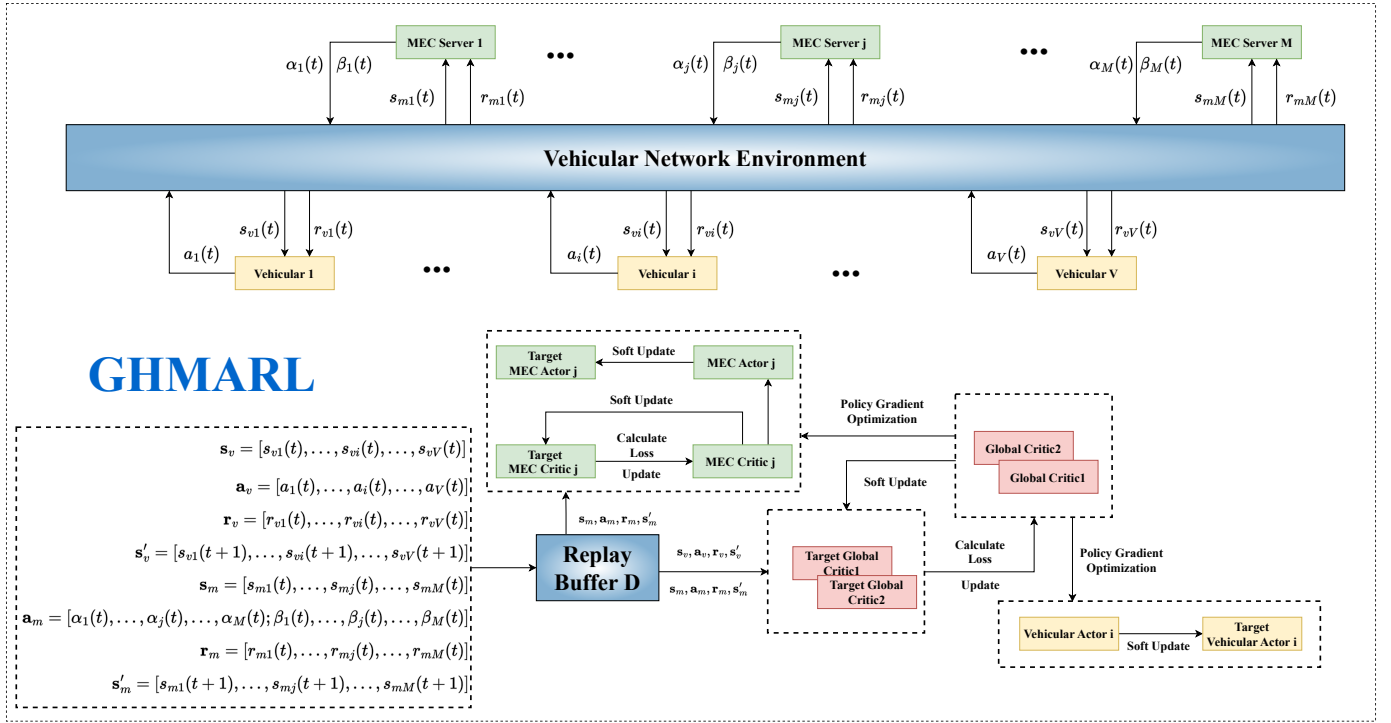


Fig. 2. The framework of the GHMRL.

global Critic uses two Critic networks, with parameters denoted as $\omega_{i,1}$ and $\omega_{i,2}$, and outputs denoted as $Q_{\omega_{i,1}}$ and $Q_{\omega_{i,2}}$, respectively. The smaller of the target network outputs $Q_{\omega'_{i,1}}$ and $Q_{\omega'_{i,2}}$ is used to calculate the target value to mitigate the problem of overestimation. Therefore, the global Critic network is updated as follows:

$$L(\omega_{i,k}) = \mathbb{E} \left[\left(Q_{\omega_{i,k}}(s_v, \mathbf{a}_v, \mathbf{s}_m, \mathbf{a}_m) - y_i^g \right)^2 \right] \quad (28)$$

where y_i^g is defined as:

$$y_i^g = r_{vi} + \gamma \cdot \min_{k=1,2} Q_{\omega'_{i,k}}(s'_v, \mathbf{a}'_v, \mathbf{s}'_m, \mathbf{a}'_m) \quad (29)$$

Similarly, the local Critic network is updated by equations (24) and (25). Thus, the policy gradient for the active agent is given by:

$$\nabla_{\theta_{vi}} \mathcal{J}_{vi} = \mathbb{E} \left[\nabla_{\theta_{vi}} \pi_{vi}(a_{vi}|s_{vi}) \nabla_{a_{vi}} Q_{\omega_{i,1}}(s_v, \mathbf{a}_v, \mathbf{s}_m, \mathbf{a}_m) \right] \quad (30)$$

The policy gradient for the passive agent is given by:

$$\begin{aligned} \nabla_{\theta_{mj}} \mathcal{J}_{mj} = & \underbrace{\mathbb{E} \left[\nabla_{\theta_{mj}} \pi_{mj}(a_{mj}|s_{mj}) \nabla_{a_{mj}} \mathbb{E} [Q_{\omega_{i,1}}(s_v, \mathbf{a}_v, \mathbf{s}_m, \mathbf{a}_m)] \right]}_{\text{GlobalCritic}} \\ & + \underbrace{\mathbb{E} \left[\nabla_{\theta_{mj}} \pi_{mj}(a_{mj}|s_{mj}) \nabla_{a_{mj}} Q_{\phi_j}(s_{mj}, a_{mj}) \right]}_{\text{LocalCritic}} \end{aligned} \quad (31)$$

It is important to note that the active agent's action a_{vi} follows a discrete distribution:

$$\mathcal{K} = (x_1, \dots, x_j, \dots, x_M) \quad (32)$$

where $x_j = P(a_{vi} = j)$ and satisfies:

$$\sum_{j=1}^M x_j = 1 \quad (33)$$

It can be observed that sampling from this discrete distribution is non-differentiable, while updating the Actor network using deterministic policy gradients requires the agent's actions to be differentiable with respect to the policy parameters. This challenge can be addressed by using the Gumbel-Softmax technique [36]. Specifically, a reparameterization factor g_j is introduced, which is noise sampled from Gumbel(0, 1) and can be expressed as:

$$g_j = -\log(-\log u), u \sim \text{Uniform}(0, 1) \quad (34)$$

Therefore, *Gumbel-softmax* sampling can be written as:

$$y_j = \frac{\exp((\log x_j + g_j)/\zeta)}{\sum_{n=1}^M \exp((\log x_n + g_n)/\zeta)}, j = 1, \dots, M \quad (35)$$

At this point, if the discrete value a_{vi} is calculated through:

$$a_{vi} = \arg \max_j y_j \quad (36)$$

this approximates the value of the discrete sampling $a_{vi} \sim \mathcal{K}$. The sampling result $\mathbf{y}$ inherently introduces a gradient with respect to $\mathbf{x}$, allowing for back-propagation in optimization tasks.

The parameter $\zeta > 0$ in the Gumbel-Softmax distribution serves as a temperature coefficient that governs the approximation of the true discrete distribution. Specifically, a higher ζ value flattens the distribution, thereby increasing the stochasticity of the sampling process and promoting exploration in decision-making. Conversely, a lower ζ value causes the generated samples to converge to a one-hot encoding of the adjusted log-probability maximum $\text{onehot}(\arg \max_j (\log \pi_j + y_j))$, facilitating deterministic selection based on the most probable action and emphasizing exploitation. Consequently, tuning this

parameter is critical for balancing exploration and exploitation during model optimization. Following the suggested settings for such distributions established in [36], ζ is fixed at 1. This value ensures effective gradient propagation while providing moderate stochasticity, a configuration proven to yield robust performance across various reinforcement learning tasks.

This expression ensures that the action selection is based on the maximum value of the adjusted logarithmic probabilities $\log x_j + y_j$, and one-hot encoding is used to select the most probable action. Therefore, by adjusting the temperature parameter, the balance between exploration and exploitation in the decision-making process can be controlled, which is critical for optimizing reinforcement learning models.

A soft update method is used to update the target networks of each agent.

$$\omega'_i \leftarrow \tau \omega_i + (1 - \tau) \omega'_i \quad (37)$$

$$\phi'_j \leftarrow \tau \phi_j + (1 - \tau) \phi'_j \quad (38)$$

$$\theta'_{vi} \leftarrow \tau \theta_{vi} + (1 - \tau) \theta'_{vi} \quad (39)$$

$$\theta'_{mj} \leftarrow \tau \theta_{mj} + (1 - \tau) \theta'_{mj} \quad (40)$$

where $\tau \in (0, 1)$ is the learning rate. Algorithm 1 presents the action execution algorithm that utilizes the Gumbel-Softmax technique. By applying the Gumbel-Softmax technique to a_{vi} , the agent is able to generate discrete actions to interact with the environment while facilitating proper gradient back-propagation. To enhance exploration during training, the active agent employs an ϵ -greedy strategy, ensuring that each action has a non-zero probability of being selected. Meanwhile, the passive agent explores by introducing noise into the chosen actions.

Algorithm 1 Execution Algorithm

Input: The weights θ_{vi} of π_{vi} , The weights θ_{mj} of π_{mj}
Output: Joint action $\mathbf{a}_v = (a_{v1}, \dots, a_{vi})$, $\mathbf{a}_m = (a_{m1}, \dots, a_{mj})$

- 1: Initialize ϵ as 0.1
- 2: Load θ_{vi} to each vehicle's Actor network
- 3: Load θ_{mj} to each MEC server's Actor network
- 4: Each vehicle observes and obtain the state s_{vi}
- 5: Each MEC server observes and obtain the state s_{mj}
- 6: **if** agent's type == vehicle **then**
- 7: **for** $i = 1, 2, \dots, V$ **do**
- 8: Compute action value a_{vi} where $a_{vi} = \pi_{vi}(s_{vi})$
- 9: **if** explore **then**
- 10: Choose a random probability $p \in [0, 1]$
- 11: $u \leftarrow \text{Random sampling in the Uniform}(0, 1)$
- 12: $g_{vi} = -\log(-\log u)$
- 13: $y_{vi} = \text{Softmax}(a_{vi} + g_{vi})$
- 14: **if** $p \leq \epsilon$ **then**
- 15: Select a random action and generate one-hot y_{vi}^{hard}
- 16: **else**
- 17: Generate one-hot representation y_{vi}^{hard} of y_{vi}
- 18: **end if**
- 19: $a_{vi} = (y_{vi}^{hard} - y_{vi}).detach + y_{vi}$
- 20: **else**
- 21: Generate one-hot of a_{vi}
- 22: **end if**
- 23: **end for**
- 24: **else if** agent's type == MEC server **then**
- 25: **for** $j = 1, 2, \dots, M$ **do**
- 26: Compute action value a_{mj} where $a_{mj} = \pi_{mj}(s_{mj})$
- 27: **if** explore **then**
- 28: $a_{mj} = a_{mj} + n_{noise}$
- 29: **end if**
- 30: **end for**
- 31: **end if**

Algorithm 2 presents the GHMARL algorithm. A batch of data is extracted for training purposes. Due to the high computational requirements and prolonged duration of the training process, it is considered to place the training process to the MEC server for deployment. In the actual execution after the training process, only the Actor networks are utilized, and each agent is capable of selecting the optimal action based on its own state.

Algorithm 2 Global Heterogeneous MARL

- 1: Start environment simulator
- 2: Initialize global critic networks
- 3: Initialize local critic networks
- 4: Initialize each agent's policy networks
- 5: **for** each episode **do**
- 6: Update vehicles locations and task information
- 7: Reset MEC server's locations and computing power
- 8: **for** each timestep t **do**
- 9: **for** each agent v_i **do**
- 10: Observe s_{vi} and select action a_{vi} using Algorithm 1
- 11: **end for**
- 12: **for** each agent m_j **do**
- 13: Observe s_{vj} get o_{mj} and get action a_{mj} using Algorithm 1
- 14: **end for**
- 15: Execute $\mathbf{a}_v, \mathbf{a}_m$ and receive rewards, $\mathbf{r}_v$ and $\mathbf{r}_m$
- 16: Store $(s_v, \mathbf{a}_v, \mathbf{r}_v, \mathbf{s}'_v, \mathbf{s}_m, \mathbf{a}_m, \mathbf{r}_m, \mathbf{s}'_m)$ in replay buffer D
- 17: **end for**
- 18: Sample minibatch from replay buffer D
- 19: **for** each agent v_i **do**
- 20: Set $y_i^g = r_{vi} + \gamma \cdot \min_{k=1,2} Q_{\omega'_{i,k}}(s'_{vi}, \mathbf{a}'_{vi}, \mathbf{s}'_{mi}, \mathbf{a}'_{mi})$
- 21: **for** k **do**
- 22: Update global critics by minimizing the loss:
- 23: $L(\omega_{i,k}) = \mathbb{E} \left[\left(Q_{\omega_{i,k}}(s_v, \mathbf{a}_v, \mathbf{s}_m, \mathbf{a}_m) - y_i^g \right)^2 \right]$
- 24: Update target parameters: $\omega'_{i,k} \leftarrow \tau \omega_{i,k} + (1 - \tau) \omega'_{i,k}$
- 25: **end for**
- 26: Update Actors by equation (30)
- 27: Update target parameters: $\theta'_{vi} \leftarrow \tau \theta_{vi} + (1 - \tau) \theta'_{vi}$
- 28: **end for**
- 29: **if** episode mod d **then**
- 30: **for** each agent m_j **do**
- 31: Set $y_j^l = r_{mj} + \gamma Q_{\phi_j^l}(s'_{mj}, \mathbf{a}'_{mj})$
- 32: Update local critics by minimizing the loss:
- 33: **end for**
- 34: **end if**
- 35: **end for**

D. Complexity Analysis

The complexity analysis consists of two related parts, that is, an analysis of the number of trainable parameters and an analysis of computational complexity the GHMARL algorithm.

The number of trainable parameters: Specifically, assume that all agents have the same observation and action spaces, denoted as ω and α respectively. The GHMARL shares a global centralized Q-function, whose parameter number can be represented as $\mathcal{O}(P(\omega + \alpha))$, where P indicates the number of agents. On the other hand, the parameter space of the local critic can be represented as $\mathcal{O}(\omega + \alpha)$.

Computational analysis: For GHMARL, the total number of neural networks is given by $2(P(C_l + A_l) + 2C_g)$, where C_l and A_l represent the number of local Critics and Actors per agent, respectively, and C_g denotes the total number of global Critics. The computational complexity for each Actor

and Critic can be represented as:

$$\mathfrak{C}^a = \sum_{i=2}^{i=L_a-1} (\Gamma_{i-1}^a \Gamma_i^a + \Gamma_i^a \Gamma_{i+1}^a) \quad (41)$$

$$\mathfrak{C}^c = \sum_{i=2}^{i=L_c-1} (\Gamma_{i-1}^c \Gamma_i^c + \Gamma_i^c \Gamma_{i+1}^c) \quad (42)$$

where L_a and L_c are the total number of layers in the Actor and Critic networks, Γ_i^a and Γ_i^c denote the number of the neurons in the i -th layer of the networks, respectively. The computational complexity of the Actor and Critic networks can be expressed as $\mathcal{O}(\mathfrak{C}^a)$ and $\mathcal{O}(\mathfrak{C}^c)$, respectively. Hence, the complexity of the proposed GHMARL framework is $\mathcal{O}(N_b \cdot E \cdot I \cdot 4C_g \cdot \mathfrak{C}_g^c) + \mathcal{O}(N_b \cdot E \cdot I \cdot (2P \cdot A_l \cdot \mathfrak{C}_l^a + 2P \cdot C_l \cdot \mathfrak{C}_l^c))$, where the indexes g and l are related to global and local neural networks, N_b is the mini-batch sampling size, E is the number of episodes, and I is the maximum training steps of each episode.

Fundamentally, the training overhead of MARL is linked to addressing environmental non-stationarity and the expansion of the joint action space [37]. The proposed GHMARL mitigates this by leveraging the advantages of the Centralized Training and Decentralized Execution (CTDE) approach. The CTDE leaves additional computational burden within the offline training phase. During execution, each agent makes independent decisions based on local observations, requiring no global information exchange or online centralized coordination [38].

To evaluate practical feasibility, Table II compares its computational complexity with benchmark algorithms. GHMARL incurs a slightly higher training complexity due to the global Critic network. However, the training is executed offline. The complexities of the online execution phase for GHMARL, MDRCO, and AC are identical, due to the fact that agents rely solely on the forward propagation of local Actor networks.

TABLE II
COMPARISON OF COMPUTATIONAL COMPLEXITY

Algorithm	Training Complexity	Execution Complexity
GHMARL	$\mathcal{O}(N_b \cdot E \cdot I \cdot 4C_g \cdot \mathfrak{C}_g^c) + \mathcal{O}(N_b \cdot E \cdot I \cdot (2P \cdot A_l \cdot \mathfrak{C}_l^a + 2P \cdot C_l \cdot \mathfrak{C}_l^c))$	$\mathcal{O}(\mathfrak{C}^a)$
MDRCO	$\mathcal{O}(N_b \cdot E \cdot I \cdot 2P \cdot (\mathfrak{C}^a + \mathfrak{C}^c))$	$\mathcal{O}(\mathfrak{C}^a)$
AC	$\mathcal{O}(N_b \cdot E \cdot I \cdot 2P \cdot (\mathfrak{C}^a + \mathfrak{C}^c))$	$\mathcal{O}(\mathfrak{C}^a)$

IV. SIMULATION RESULTS

This section presents an evaluation of the performance of the proposed GHMARL via extensive simulations. The key parameters and their default settings are presented in Table III.

Each episode is set to have 60 time steps, and the training is conducted for 5000 episodes with an evaluation every 100 episodes. During the evaluation, each agent uses its Actor network to take action. In this study, the value of the objective function is utilized as a criterion to evaluate the effectiveness of the algorithm.

TABLE III
SIMULATION PARAMETERS AND SETTINGS

Parameter	Value
<i>Environment Parameters</i>	
RSU-MEC inter-site distance (ISD)	100 m
Number of Vehicles V	5 – 25
Number of MEC servers M	5
Vehicle speed	120 km/h
Maximum task data size x_i	1000 KB
Maximum task computational complexity	4000
Mean of the Poisson distribution	1/2
Vehicle transmission power P_t	30 dBm
Noise power σ^2	−114 dBm
Maximum bandwidth of MEC server W_M	2 MHz
Capability of MEC server F_M	6 GHz
Channel gain c	0.25
Constant coefficient w_0	0.0001
<i>Neural Networks Parameters</i>	
Experience replay buffer size	100000
Mini batch size	64
Local Actor networks hidden layers (Num/Size)	2 / 1024, 512
Local Critic networks hidden layers (Num/Size)	2 / 512, 256
Global Critic hidden layers (Num/Size)	3 / 1024, 512, 256
Critic / Actor networks learning rate	0.001 / 0.0001
Discount factor γ	0.99
Target networks soft update parameter τ	0.0005

Given the distinct roles of vehicles and MEC servers, modeling them as heterogeneous agents is essential for practical VEC scenarios. It is worth noting that most existing MARL schemes in VEC studies treat only vehicles or MEC servers as agents, which is incompatible with our heterogeneous setting. Therefore, to ensure a fair assessment of GHMARL, we select two representative baseline schemes compatible with the heterogeneous setting. They are MDRCO [1], a state-of-the-art scheme explicitly addressing vehicle-server interactions, and AC [39], a classic heterogeneous agent-based algorithm.

- 1) MDRCO (Multi-agent DRL scheme for Computation Offloading) [1]: In this scheme, heterogeneous agents are employed, that is, MEC servers are modeled as agents that train resource allocation strategies, while vehicles as agents making offloading decisions, choosing the nearest MEC server. However, MDRCO does not consider reserving resources, nor does it utilize a specialized architecture to manage the collaborative/competitive dynamics between these heterogeneous agents.
- 2) AC (Actor-Critic) [39]: In this scheme, vehicles and MEC servers function as distinct heterogeneous agents, and the resource reservation is considered. However, compared to the proposed GHMARL, the AC scheme does not establish a clear distinction between active and passive agents, nor does it deploy dedicated Global and Local Critic networks tailored to the divergent objectives of each agent type.

A. Convergence of the GHMARL algorithm

The Actor-Critic method is a framework that facilitates the refinement of the policy network through the value network. However, the reliability of the value network is a critical factor in the accuracy of its action evaluations, which, in turn determines the efficacy of updates to the policy network. When the value network exhibits instability, rapid update of the policy network may destabilize the value network training due to the significant alterations in the policy network. Consequently, the update rate of the policy network and the target network should be moderated to be less than that of the value network.

The fundamental principle underlying TD3 is to periodically refresh the passive agent's policy network and target network every d training steps. Consequently, the selection of an appropriate value for d is crucial for the convergence of the algorithm. To fully leverage the delayed update mechanism of TD3, d must be a positive integer and greater than 1, as when $d = 1$, the algorithm degrades to standard DDPG, where the Actor updates synchronously with the Critic at every step. Such frequent adjustments, driven by premature Critic evaluations, inevitably trigger overestimation bias and policy oscillation. Therefore, $d = 2$ serves as the minimum value for implementing the TD3 mechanism. Furthermore, the impact of different values of d is studied and demonstrated in Fig. 3 for GHMARL, where an environment comprising 15 vehicles is considered. The results indicate that larger values of d lead to performance degradation and slower convergence. Hence, $d = 2$ is considered as the optimal setting and used in the subsequent simulations to ensure the optimal performance of the algorithm.

Fig. 4 presents a comparison of convergence under different average vehicle velocities for the GHMARL algorithm. The velocity of each vehicle follows a truncated normal distribution (greater than 0) with a standard deviation of 10 km/h, and a mean of $v = 30, 60, 80, 100$, and 120 km/h respectively.

The number of vehicles is set to 10. As demonstrated, the algorithm can achieve rapid convergence at all vehicle velocities, particularly at higher velocities, demonstrating good practicality and resilience of the algorithm.

B. Overall performance comparison

The system reward values of various algorithms is presented in Fig. 5, considering different numbers of vehicles. It is shown that the proposed GHMARL can achieve significantly higher reward values, in comparison with the AC and MDRCO algorithms. This observation underscores its resilience in a multi-agent environment. This is due to the fact that GHMARL has designed two distinct reward functions, which comprehensively consider the influence of heterogeneous agents on training stability and performance. Furthermore, GHMARL trains agents on MEC server data center without the need to retain global information, enhancing the stability and efficiency of the training process.

In Fig. 6 and 7, the performances of different algorithms are compared in terms of task delay and computation resource usage versus varying vehicle-to-MEC server ratios. This ratio

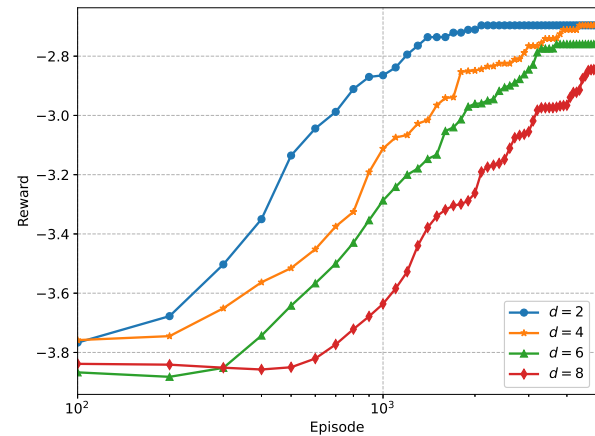


Fig. 3. Convergence of the GHMARL algorithm with different d 's.

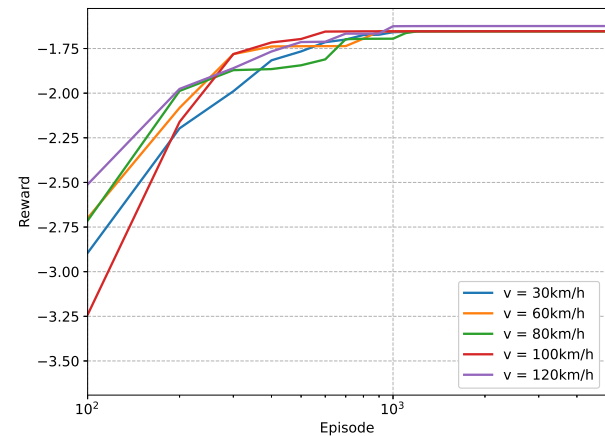


Fig. 4. Convergence of the GHMARL algorithm with different vehicle velocities.

ranges from 1 to 5 over a 500m road segment with 5 RSUs, representing the vehicle density range of 10 to 50 vehicles/km. This range reflects medium to congested traffic levels in typical IoV systems. Compared to MDRCO algorithm, at low vehicle-to-MEC server ratios, the delay performance of GHMARL is similar, due to the fact that there exist adequate MEC resources to serve the tasks. When the ratio increases, the MEC servers have to take more task loads, leading to constrained computational resources. At high ratios, GHMARL exhibits slightly higher delay, however, its computational resource usage is substantially lower. On the other hand, compared to AC algorithm, GHMARL outperforms significantly in terms of delay. At lower ratios, both algorithms utilize comparable computational resources. However, at high ratios, GHMARL's resource usage is significantly lower. In summary, for GHMARL, the design of local Critic networks for MEC servers allows it to meet vehicle service demands while minimizing resource usage. In addition, the consideration of reserving resources for future demand significantly improves GHMARL's resource

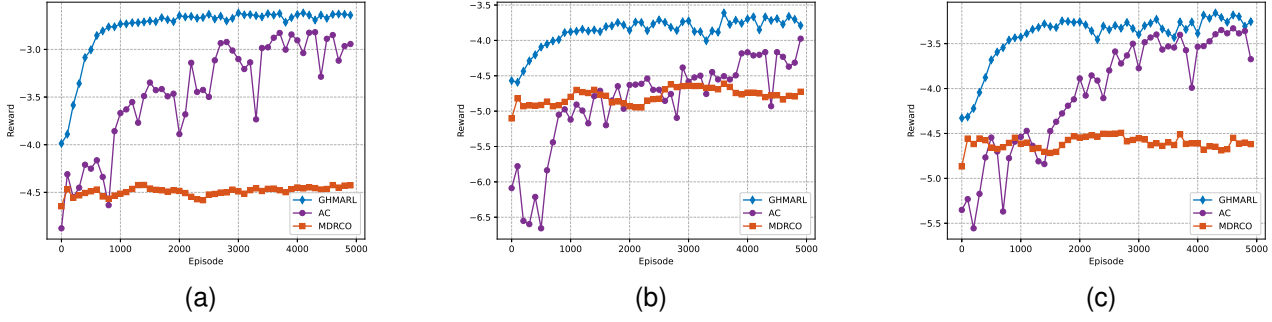


Fig. 5. The system reward with different number of vehicles.(a) Number of vehicles = 10. (b) Number of vehicles = 15. (c) Number of vehicles = 20.

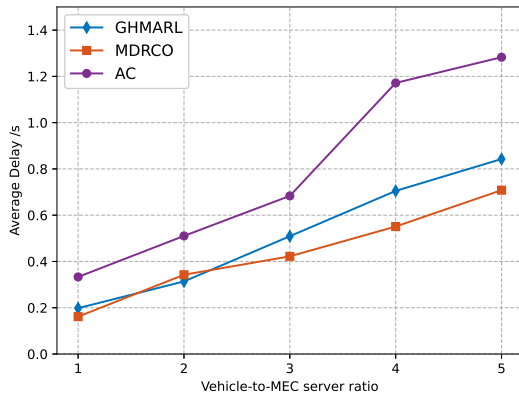


Fig. 6. Average delay versus vehicle-to-MEC server ratio.

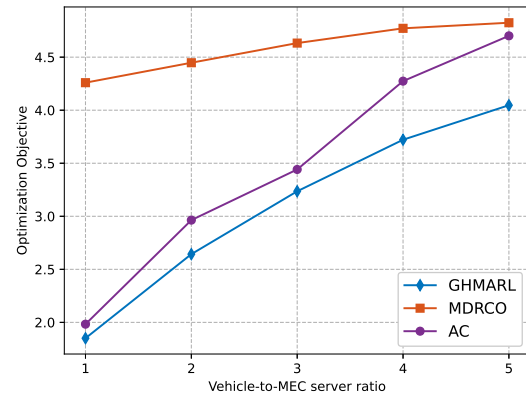


Fig. 8. Optimization result for $P1$ versus vehicle-to-MEC server ratio.

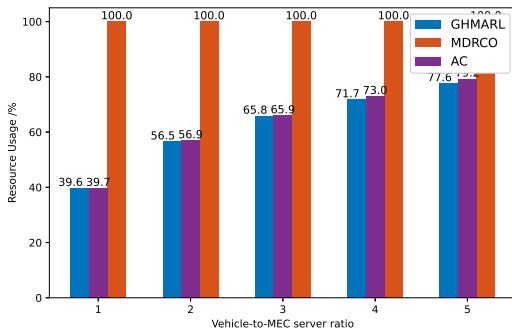


Fig. 7. Resource consumption versus vehicle-to-MEC server ratio.

utilization and system flexibility. In contrast, MDRCO does not consider resource reservation, leading to less flexibility in MEC server resource allocation.

Fig. 8 shows the optimization result for $P1$ versus vehicle-to-MEC server ratio. It can be seen that as the ratio increases, the performance disparity between GHMARL and MDRCO gradually diminishes. However, GHMARL exhibits consistent superiority over both MDRCO and AC. This highlights that the proposed GHMARL can achieve optimized balance between maximizing resource utilization and minimizing delay, demonstrating superior robustness and adaptability to the

environment dynamics.

V. CONCLUSIONS

In this paper, a novel GHMARL algorithm was proposed to address the computational offloading and resource allocation optimization issue for computationally intensive vehicle services in vehicular edge networks. The GHMARL is an enhancement of the conventional MARL framework. It defines two distinct agent classes and implements different designs in the action and state spaces accordingly. Furthermore, GHMARL implements the global and local reward functions for the two agent classes respectively. The objective of this novel design is to ensure an optimal balance between task delay and resource consumption. In addition, GHMARL considers to reserve resources for MEC servers for future service demand, which improves the system flexibility in resource usage. Extensive simulation results demonstrate that GHMARL outperforms the benchmarking MDRCO and AC algorithms, in terms of achieving maximum system reward, minimizing resource consumption while meeting delay requirements. In addition, GHMARL is able to achieve rapid convergence and demonstrates performance consistency under environment dynamics. To further enhance its practical applicability, future work will extend the theoretical framework by considering factors such

as heterogeneous task types, vehicle energy constraints and MEC queuing mechanism.

REFERENCES

- [1] X. Zhu, Y. Luo, A. Liu, M. Z. A. Bhuiyan, and S. Zhang, "Multiagent deep reinforcement learning for vehicular computation offloading in IoT," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9763–9773, 2021.
- [2] A. Yang, J. Weng, K. Yang, C. Huang, and X. Shen, "Delegating authentication to edge: A decentralized authentication architecture for vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 1284–1298, 2022.
- [3] X. Liu, H. Song, and A. Liu, "Intelligent UAVs trajectory optimization from space-time for data collection in social networks," *IEEE Trans. Neww. Sci. Eng.*, vol. 8, no. 2, pp. 853–864, 2021.
- [4] S. Misra and S. Bera, "Soft-VAN: Mobility-aware task offloading in software-defined vehicular network," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 2071–2078, 2020.
- [5] V. H. Hoang, T. M. Ho, and L. B. Le, "Mobility-aware computation offloading in MEC-based vehicular wireless networks," *IEEE Commun. Lett.*, vol. 24, no. 2, pp. 466–469, 2020.
- [6] A. Yang, J. Weng, N. Cheng, J. Ni, X. Lin, and X. Shen, "DeQoS attack: Degrading quality of service in VANETs and its mitigation," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 4834–4845, 2019.
- [7] Y. Ren, T. Wang, S. Zhang, and J. Zhang, "An intelligent big data collection technology based on micro mobile data centers for crowdsensing vehicular sensor network," *Pers. Ubiquitous Comput.*, vol. 27, no. 3, pp. 563–579, 2023.
- [8] S. Guo, J. Liu, Y. Yang, B. Xiao, and Z. Li, "Energy-efficient dynamic computation offloading and cooperative task scheduling in mobile cloud computing," *IEEE Trans. Mobile Comput.*, vol. 18, no. 2, pp. 319–333, 2019.
- [9] Z. Kuang, L. Li, J. Gao, L. Zhao, and A. Liu, "Partial offloading scheduling and power allocation for mobile edge computing systems," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6774–6785, 2019.
- [10] Y. Xu, J. Ren, G. Wang, C. Zhang, J. Yang, and Y. Zhang, "A blockchain-based nonrepudiation network computing service scheme for industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 15, no. 6, pp. 3632–3641, 2019.
- [11] S. Zhang, G. Wang, M. Z. A. Bhuiyan, and Q. Liu, "A dual privacy preserving scheme in continuous location-based services," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 4191–4200, 2018.
- [12] S.-W. Ko, K. Han, and K. Huang, "Wireless networks for mobile edge computing: Spatial modeling and latency analysis," *IEEE Trans. Wireless Commun.*, vol. 17, no. 8, pp. 5225–5240, 2018.
- [13] P. Wang, Z. Zheng, B. Di, and L. Song, "HetMEC: Latency-optimal task assignment and resource allocation for heterogeneous multi-layer mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 18, no. 10, pp. 4942–4956, 2019.
- [14] M. Kato, T. K. Rodrigues, and N. Kato, "Breakout local search solution to the offloading decision problem in a multi-access edge computing cloud-enabled network," *IEEE Trans. Emerg. Topics Comput.*, vol. 13, no. 3, pp. 1328–1338, 2025.
- [15] Z. Gao, L. Yang, and Y. Dai, "Large-scale computation offloading using a multi-agent reinforcement learning in heterogeneous multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3425–3443, 2023.
- [16] X. Zhu, Y. Luo, A. Liu, W. Tang, and M. Z. A. Bhuiyan, "A deep learning-based mobile crowdsensing scheme by predicting vehicle mobility," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4648–4659, 2021.
- [17] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint offloading and resource allocation in vehicular edge computing and networks," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*. IEEE, 2018, pp. 1–7.
- [18] Z. Wang, Z. Zhong, D. Zhao, and M. Ni, "Vehicle-based cloudlet relaying for mobile computation offloading," *IEEE Trans. Veh. Technol.*, vol. 67, no. 11, pp. 11 181–11 191, 2018.
- [19] Z. Ning, K. Zhang, X. Wang, L. Guo, X. Hu, J. Huang, B. Hu, and R. Y. Kwok, "Intelligent edge computing in Internet of Vehicles: A joint computation offloading and caching solution," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2212–2225, 2021.
- [20] M. Tang and V. W. Wong, "Deep reinforcement learning for task offloading in mobile edge computing systems," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 1985–1997, 2022.
- [21] M. Z. Alam and A. Jamalipour, "Multi-agent DRL-based Hungarian algorithm (MADRLHA) for task offloading in multi-access edge computing Internet of Vehicles (IoVs)," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 7641–7652, 2022.
- [22] L. Ma, H. Shi, J. Li, and K.-S. Hwang, "A multi-agent reinforcement learning based offloading strategy for multi-access edge computing," in *Proc. Int. Autom. Control Conf. (CACIS)*. IEEE, 2021, pp. 1–5.
- [23] Z. Sun, Y. Mo, and C. Yu, "Graph-reinforcement-learning-based task offloading for multiaccess edge computing," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3138–3150, 2023.
- [24] L. Zhao, E. Zhang, S. Wan, A. Hawbani, A. Y. Al-Dubai, G. Min, and A. Y. Zomaya, "MESON: A mobility-aware dependent task offloading scheme for urban vehicular edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4259–4272, 2024.
- [25] M. I.-C. Wang, C. H.-P. Wen, and H. J. Chao, "Hierarchical cooperation and load balancing for scalable autonomous vehicle routing in multi-access edge computing environment," *IEEE Trans. Veh. Technol.*, vol. 72, no. 6, pp. 6959–6971, 2023.
- [26] W. Duan, X. Gu, M. Wen, Y. Ji, J. Ge, and G. Zhang, "Resource management for intelligent vehicular edge computing networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 9797–9808, 2022.
- [27] P. Xiang, H. Shan, M. Wang, Z. Xiang, and Z. Zhu, "Multi-agent RL enables decentralized spectrum access in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 10, pp. 10 750–10 762, 2021.
- [28] M. Parvini, M. R. Javan, N. Mokari, B. Abbasi, and E. A. Jorswieck, "AoI-aware resource allocation for platoon-based C-V2X networks via multi-agent multi-task reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 72, no. 8, pp. 9880–9896, 2023.
- [29] J. Dong, S. Gao, H. Lu, Y. Cao, Y. Yu, and X. Ma, "Joint optimization of resource allocation and tasks scheduling in network slicing enabled Internet of Vehicles," in *Proc. IEEE 8th Int. Conf. Comput. Commun. (ICCC)*. IEEE, 2022, pp. 552–556.
- [30] G. Cui, Y. Long, L. Xu, and W. Wang, "Joint offloading and resource allocation for satellite assisted vehicle-to-vehicle communication," *IEEE Syst. J.*, vol. 15, no. 3, pp. 3958–3969, 2021.
- [31] Y. Xiao, Y. Song, and J. Liu, "Collaborative multi-agent deep reinforcement learning for energy-efficient resource allocation in heterogeneous mobile edge computing networks," *IEEE Trans. Wireless Commun.*, 2023.
- [32] J. Chen, H. Xing, Z. Xiao, L. Xu, and T. Tao, "A DRL agent for jointly optimizing computation offloading and resource allocation in MEC," *IEEE Internet Things J.*, vol. 8, no. 24, pp. 17 508–17 524, 2021.
- [33] S. Nath and J. Wu, "Deep reinforcement learning for dynamic computation offloading and resource allocation in cache-assisted mobile edge computing systems," *Intell. Conver. Netw.*, vol. 1, no. 2, pp. 181–198, 2020.
- [34] Y. Lin, L. Xiao, Y. Tao *et al.*, "Multi-agent computing-energy-efficiency optimization in vehicular edge computing: Non-cooperative versus co-operative solutions," *IEEE Trans. Wireless Commun.*, vol. 24, no. 7, pp. 5461–5476, 2025.
- [35] M. Chen, A. Liu, N. N. Xiong *et al.*, "ANNS: An intelligent advanced non-convex non-smooth scheme for IRS-aided next generation mobile communication networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 9, pp. 8959–8973, 2025.
- [36] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel-Softmax," *arXiv preprint arXiv:1611.01144*, 2016.
- [37] F. Christianos, L. Schäfer, and S. Albrecht, "Shared experience actor-critic for multi-agent reinforcement learning," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 10 707–10 717, 2020.
- [38] C. Amato, "An introduction to centralized training for decentralized execution in cooperative multi-agent reinforcement learning," *arXiv preprint arXiv:2409.03052*, 2024.
- [39] H. Yang, X. Xie, and M. Kadoch, "Intelligent resource management based on reinforcement learning for ultra-reliable and low-latency IoV communication networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 4157–4169, 2019.



Huilin Yang is currently pursuing the M.S. degree with the Faculty of Electrical Engineering and Computer Science, Ningbo University, Ningbo, China. Her research interests include Internet of Vehicles, semantic communication, and reinforcement learning.



Xinjie Yang is currently an Associate Professor with the Faculty of Electrical Engineering and Computer Science, Ningbo University, Ningbo, China. His research interests include next-generation mobile communication system architectures, cellular Internet of Things access technologies, and the performance of Internet of Vehicles networks.



Jiazhi Yang received the M.S. degree from the Faculty of Electrical Engineering and Computer Science, Ningbo University, Ningbo, China. His research interests include Internet of Vehicles, mobile edge computing, computation offloading, and reinforcement learning.



Ruizhe Li is currently pursuing the M.S. degree with the Faculty of Electrical Engineering and Computer Science, Ningbo University, Ningbo, China. His research interests include Internet of Vehicles, integrated sensing and communication, mobile edge computing, and reinforcement learning.



Nan Ma (Member, IEEE) is a Professor with the Faculty of Information and Communication Engineering, Beijing University of Posts and Telecommunications, Beijing, China, where he is also the Director of the Beijing Key Laboratory of Wireless Communications and Testing Technology. His research interests include wireless communication theory, testing technology, and big data applications.



Pei Xiao (Senior Member, IEEE) is a Professor in Wireless Communications in the Institute for Communication Systems (ICS) at University of Surrey. He is currently the technical manager of 5GIC/6GIC, leading the research team in the new physical layer work area, and coordinating/supervising research activities across all the work areas (<https://www.surrey.ac.uk/institute-communication-systems/5g-6g-innovation-centre>). Prior to this, he worked at Newcastle University and Queen's University Belfast. He also held positions at Nokia Networks in Finland. He has published extensively in the fields of communication theory, RF and antenna design, signal processing for wireless communications, and is an inventor on over 15 recent patents addressing bottleneck problems in 5G/6G systems.

Affiliation: Institute for Communication Systems (ICS), Home for 5GIC & 6GIC, University of Surrey, Guildford, Surrey, GU2 7XH, United Kingdom.
Email: p.xiao@surrey.ac.uk